Group-Fused Apply Projection (pair_proj_fused)

Oct 23, 2026 · @Jung Yoon

Overview

Fusing each `pair_proj_grouped` group into one thread cuts the projection kernel from 707 ms to 465 ms per tree (1.52×) and the whole tree from 1,159 ms to 934 ms (−19%) on HIGGS1 at depth 10.

`pair_proj_grouped` gives every node 2N projections for N features: N axis-aligned ones and N random 2-feature pairs. It orders them so each pair sits right after the axis-aligned projection of its lower feature. Every projection in a group therefore reads the same column `f`.

The unfused DRSW kernel still runs one thread per (row, projection), so a row's `x_f`, node id and node offset are loaded once for every projection in the group. `--config=pair_proj_fused` runs one thread per (row, group) instead: it loads those values once and computes the whole group.

The output is the same projected buffer, in the same layout, summed in the same order, so nothing downstream changes. Setup for all numbers: HIGGS1 (11,000,000 rows × 28 features), L4 GPU, seed 1, sampling without replacement at ratio 1.0, non-symmetric projections, node-id DRSW path.

Grouped projection layout

Per node, the 2N projection slots run feature by feature: `axis(f)`, then every random pair whose lower feature is `f`, in ascending pair index. Group `f` therefore always starts with the only 1-entry projection in it.

The GPU sampler (`SampleProjectionCountKernel` / `SampleProjectionFillKernel` under `PAIR_PROJ_GROUPED`) builds this with the same curand draws as `pair_proj`, so the candidate set is unchanged. Each pair is stored sorted, `[lo, hi]`, so its first entry is the group's anchor column.

Worked example: N = 4 features, one node

The node draws four pairs: j0 = (1, 3), j1 = (0, 2), j2 = (2, 3), j3 = (0, 1). Grouping them under their lower feature gives:

Slot	Projection	Entries (col)	Count	Group (anchor)
0	axis(0)	0	1	0
1	pair j1	0, 2	2	0
2	pair j3	0, 1	2	0
3	axis(1)	1	1	1
4	pair j0	1, 3	2	1
5	axis(2)	2	1	2
6	pair j2	2, 3	2	2
7	axis(3)	3	1	3

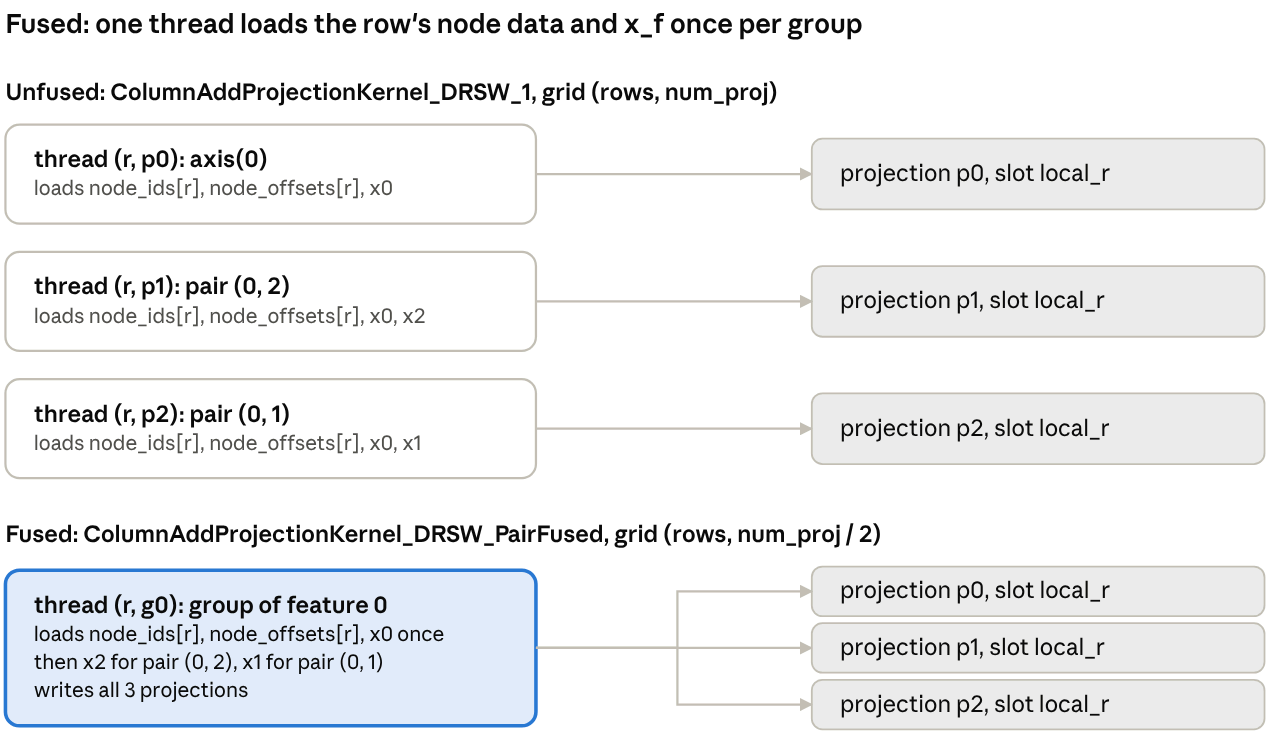
The CSR arrays for this node are then:

Array	Values
`d_offset` (prefix sum of counts, per segment)	0, 1, 3, 5, 6, 8, 9, 11, 12
`d_flat_projection_col_idx`	0 · 0 2 · 0 1 · 1 · 1 3 · 2 · 2 3 · 3
`d_flat_projection_weights`	1.0 × the feature's norm factor, every entry
group start (built by the fused path)	0, 3, 5, 7, 8

Group sizes vary by feature and by node: low-numbered features anchor more pairs, and the last feature never anchors one. Expected group size for feature `f` is 1 + 2(N − 1 − f)/(N − 1), from 3 at f = 0 down to 1 at f = N − 1.

Thread mapping

The fused kernel replaces k threads per (row, group) with one, so the shared loads happen once instead of k times.



Same outputs, same order of terms, so projected values are bit-identical to the unfused kernel

thread mapping for group 0 of the worked example · row r

Group 0 of the worked example holds axis(0), pair (0, 2) and pair (0, 1). Unfused, three threads in three `blockIdx.y` planes each reload the row's node id, offset and `x0`. Fused, one thread loads them once and writes the same three slots.

Kernels

The fused path is two launches in `oblique_apply_projection.cu`, inside `NodeIdDRSW_ProjectionPairFused_wrap`, which `training.cc` calls in place of `NodeIdDRSW_ProjectionOnly_wrap` when `PAIR_PROJ_FUSED` is defined.

1. `PairProjFused_GroupStartKernel`

One thread per node scans the node's `num_proj` segments and records every 1-entry segment as a group start:

```
int* gs = group_start + (long long)node * (num_groups + 1);
int g = 0;
for (int s = 0; s < num_proj && g < num_groups; ++s) {
    const int seg = node * num_proj + s;
    if (col_offset[seg + 1] - col_offset[seg] == 1) gs[g++] = s;
}
for (; g <= num_groups; ++g) gs[g] = num_proj;   // terminator
```

`num_groups = num_proj / 2` (one per feature). The buffer holds `num_nodes × (num_groups + 1)` ints, is allocated once in a static and grown on demand. Cost is `num_nodes` threads × 56 iterations on HIGGS, negligible next to the projection.

2. `ColumnAddProjectionKernel_DRSW_PairFused`

Grid `(ceil(num_rows / 256), num_groups)`, 256 threads per block; `blockIdx.y` is the group `g`. Each thread grid-strides over dataset rows; the core loop per (row, group) is:

```
const int anchor_col = flat_col_data[col_offset[seg0]];
const float x_anchor = dataset[(size_t)anchor_col * num_rows + r];
const long long base = (long long)row_start * num_proj + node_offset;
for (int p = p_begin; p < p_end; ++p) {
    const int seg = node_id * num_proj + p;
    float sum = 0.0f;
    for (int idx = col_offset[seg]; idx < col_offset[seg + 1]; ++idx) {
        const int col = flat_col_data[idx];
        const float x = (col == anchor_col)
            ? x_anchor : dataset[(size_t)col * num_rows + r];
        sum += flat_weights[idx] * x;
    }
    projected[base + (long long)p * rows_node] = sum;
}
```

Step by step, each thread:

1. Reads `node_id = node_ids[r]`; skips rows outside the bag (`-1`).
2. Reads `row_start`, `rows_node` from `node_row_off` and `node_offset = node_offsets[r]` — **once per group**.
3. Reads the group's slot range `[gs[g], gs[g+1])` for this node.
4. Reads the anchor column from the group's first entry and loads `x_anchor = dataset[anchor_col·num_rows + r]` — **once per group**.

5. For each projection `p` in the group, sums `w·x` over its entries, using `x_anchor` when `col == anchor_col` and loading the column otherwise, then writes `projected[row_start·num_proj + p·rows_node + node_offset]`.

Entries are summed in the same order as `ColumnAddProjectionKernel_DRSW_1`, so each projected value is bit-identical to the unfused kernel's. The anchor test compares columns rather than assuming entry 0 is the anchor, so the kernel stays correct if the pair order ever changes. With `num_proj < 2` (one feature) the wrapper falls back to the unfused kernel.

Memory traffic per row

Fusing cuts the per-row global accesses of the projection by a third on HIGGS, from 252 to 168, which matches the measured 0.66× projection time per tree.

For one group with `m` pairs (k = 1 + m projections), counting 4-byte accesses that go to per-row arrays. Small per-node arrays (`node_row_off`, `col_offset`, the CSR entries) are cache-resident in both kernels and left out.

Access per row	Unfused, per group	Fused, per group
`node_ids[r]`	k	1
`node_offsets[r]`	k	1
dataset values	1 + 2m	1 + m
projected writes	k	k

Per row per node on HIGGS (N = 28, so 28 groups, 28 axis + 28 pair projections, m summing to 28):

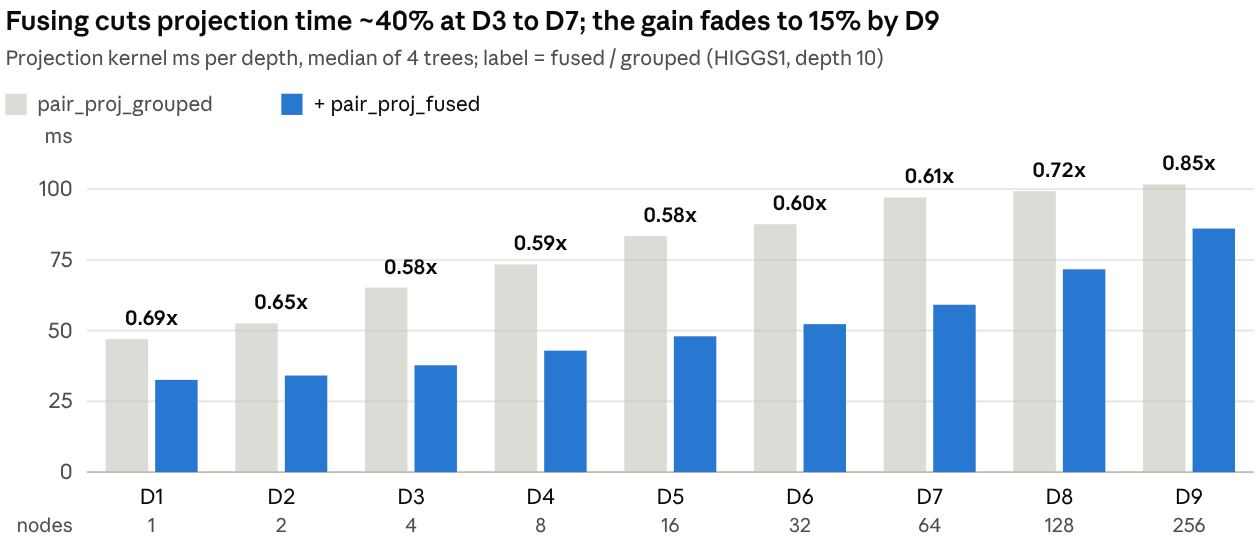
Access per row	Unfused	Fused
`node_ids[r]`	56	28
`node_offsets[r]`	56	28
dataset values	84	56
projected writes	56	56

Access per row	Unfused	Fused
Total	**252 (1,008 B)**	**168 (672 B)**

The fused/unfused ratio is 168 / 252 = 0.67, against a measured 465 / 707 = 0.66. That fit suggests the kernel is bound by these accesses, but the kernel has not been profiled. Reads of the same row by different projections in the unfused kernel may partly hit L2, which this count ignores.

Measured results

Fused projection is 1.52× faster per tree, and the gain is largest at mid depths. Both builds: `bfs_only + use_gpu + node_id_drsw + offsets_only + scattered_label_hist + pair_proj_grouped`, the second adding `pair_proj_fused`; HIGGS1, depth 10, 2 trees × 2 alternating repeats.



HIGGS1 benchmark runs, 2026-10-23 · 2 trees × 2 alternating repeats per build

The gain fades past D7, probably because a warp's 32 rows then span many nodes, each with its own groups and columns, so warps diverge and dataset reads stop coalescing. D10 only finalizes leaves and has no projection.

Per tree

Stage	Grouped (ms)	Fused (ms)	Change
Projection kernel	707	465	0.66× (−242 ms)
Split finding (`eval_proj`)	400	418	+4.5% (+18 ms)
Tree total	1,159	934	−19% (−225 ms)

Repeats agreed within 1 ms. A rerun of the fused build from `main` gave 466 ms and 938 ms. Per-depth node counts were identical in every run, but every node split to 512 leaves at D10, so that check is weak.

Limits, open questions and build config

The fused path covers only the node-id DRSW projection; everything else is unchanged.

Limits

- Not fused: `ColumnAddProjectionKernel_SRDW_1_2D` (baseline and `nid_srdw_proj`) and the `PROJ_COMPARE_AB` diagnostic, which still call the unfused kernels.
- Uneven work: group `f` holds about $1 + 2(N - 1 - f)/(N - 1)$ projections, so `blockIdx.y = 0` planes do about 3× the work of the last plane.
- Group detection relies on axis projections being the only 1-entry projections, which holds for $N \geq 2$.

Open questions

- [ ] Why did split finding rise 4.5% (400 → 418 ms) when nothing downstream changed?
- [ ] Does `--config=symmetric_proj` recover the D8–D9 gain, as it did for plain DRSW?
- [ ] Check bit-identical outputs directly (compare projected buffers or split thresholds); node counts alone are a weak check at depth 10.
- [ ] Would fusing the whole node (all 2N projections per thread) beat per-group fusion?

Build

```
--config=bfs_only --config=use_gpu --config=node_id_drsw \
--config=offsets_only --config=scattered_label_hist \
--config=pair_proj_grouped --config=pair_proj_fused
```

An `#error` in `training.cc` requires `PAIR_PROJ_GROUPED` and `NODE_ID_DRSW`; the code is uncommitted on `main`.